

Addressing Optimism Bias in Sequence Modeling for RL (SPLT Transformer)

백승언

22 Oct, 2023

- **Introduction**

- Limitations in previous offline reinforcement learning

- **SPLT Transformer**

- Sampling-based planning algorithm
- SPLT Transformer

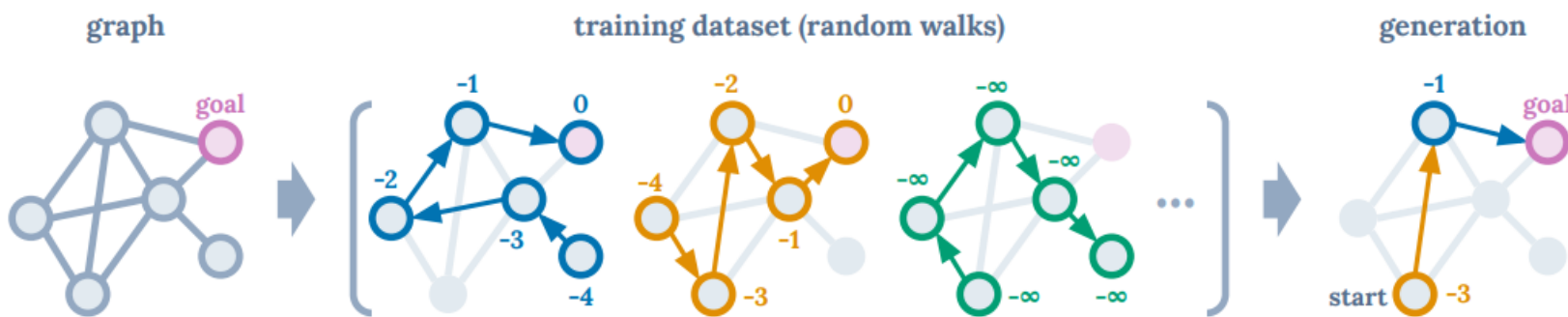
- **Experiments**

- Environments
- Results

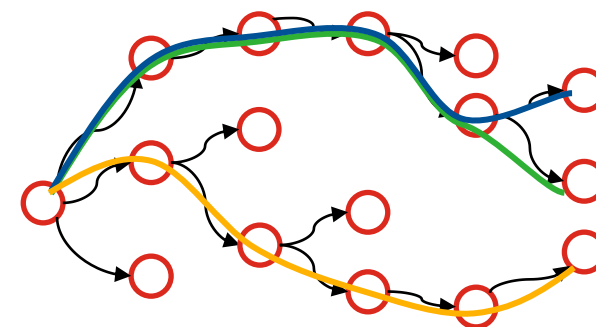
Introduction

Limitations in previous offline reinforcement learning

- Most prior works in offline RL have focused on the mainly deterministic D4RL benchmarks and weakly stochastic Atrai benchmarks
 - Therefore, there has been limited focus on the difficulties of deploying such methods in largely stochastic domains such as autonomous driving, transportation, and finance
- Recently, some works have explored leveraging high-capacity sequence models in sequential decision-making problems.
 - However, these methods focused on deterministic env and utilized naïve action selection techniques.
 - The authors supposed that it could lead to overly aggressive and optimistic behavior



Return query in Decision Transformer(DT)



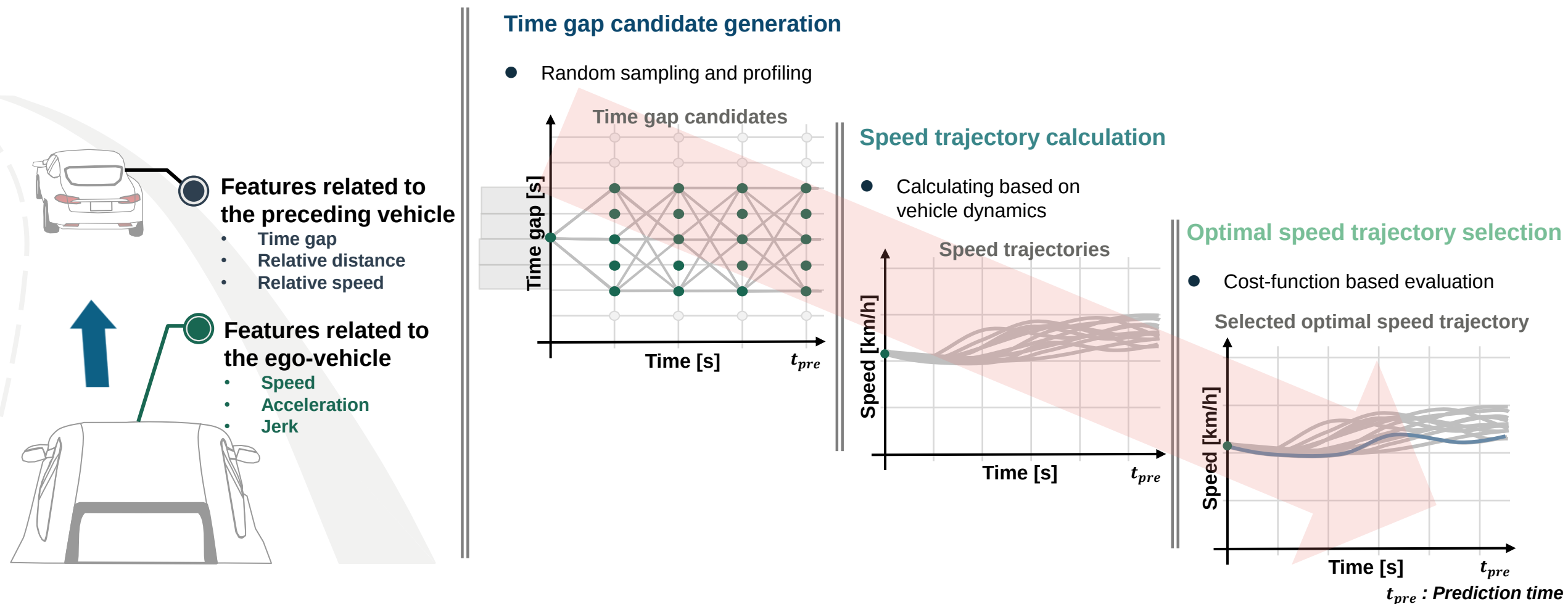
Beam search in Trajectory Transformer(TT)

SPLT Transformer

Sampling-based planning algorithm

● Process of the conventional speed planning algorithm in autonomous vehicle

- Sampling-based planning algorithm using time gap distribution



● Overview of the SPLT Transformer

- Existing offlineRL algorithms have generally been applied to deterministic/weakly stochastic environments different largely from the real-world
 - D4RL benchmark, Atari benchmark, and so on
- The proposed model is designed with a separated transformer-based VAE model for predicting the action, observation, reward, and discounted return
 - Transformer-based encoders encode the transition history for policy decoder and world model decoder
 - The policy decoder estimates the next action depends on the action excepted transition history
 - The world model decoder estimates the observation, reward, and discounted return
- Additionally, they enhanced the planning technique for offlineRL as a sequence modeling method for addressing the optimistic/sub-optimal behavior
 - They utilized a sampling-based planning technique which is the selection of the best trajectory within the generated candidate trajectory set
- Evaluation demonstrated that SPLT Transformer has outperformed in self-driving tasks which has a large stochasticity in terms of success ratio and generalization performance

SPLT Transformer (II) – Architecture

● SeParated Latent Trajectory Transformer(SPLT Transformer)

- They designed the separated Transformer-based discrete latent variable VAEs to represent policy and world models

▪ Encoders

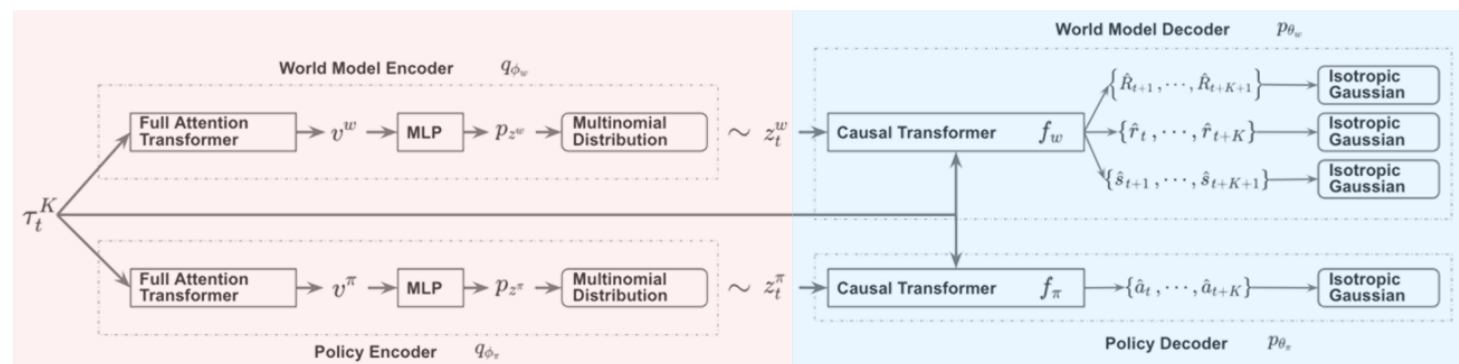
- Both the world encoder q_{ϕ_w} and policy encoder q_{ϕ_π} use the same architecture(**non masking GPT architecture**) and receive the same trajectory τ_t^K
 - $\tau_t^K = \{s_t, a_t, s_{t+1}, a_{t+1}, \dots, s_{t+K}, a_{t+K}\}$
- These encoders output a n_w or n_π dimensional discrete latent variable with each dimension having c possible values
 - $z_t^w \sim q_{\phi_w}(\cdot | \tau_t^K), z_t^w \in \{1, \dots, c\}^{n_w}$
 - $z_t^\pi \sim q_{\phi_\pi}(\cdot | \tau_t^K), z_t^\pi \in \{1, \dots, c\}^{n_\pi}$

▪ Policy decoder

- The policy decoder uses a similar input trajectory representation and use the **causal Transformer**
 - $\tau_t'^k = \{s_t, a_t, s_{t+1}, a_{t+1}, \dots, s_{t+k}\}$
- Then, the policy decoder takes the latent variable z_π , and output mean of policy distribution which is assumed with isotropic Gaussian
 - $p_{\theta_\pi}(a_{t+k} | \tau_t'^k; z^\pi) := \mathcal{N}(f_\pi(\tau_t'^k, z^\pi), I)$

▪ World model decoder

- The world model decoder is very similar to policy decoder, except that its goal is to estimate
 - $p_{\theta_w}(s_{t+k+1} | \tau_t^k; z^w), p_{\theta_w}(r_{t+k} | \tau_t^k; z^w), p_{\theta_w}(R_{t+k+1} | \tau_t^k; z^w),$
where $\forall k \in [1, K]$
- The world model decoder is similarly represented with a causal Transformer and incorporates its latent variable z^w and output unit-variance isotropic Gaussian dist.
 - $p_{\theta_w}(\phi_{t+k+1} | \tau_t^k; z^w) := \mathcal{N}(f_w^\phi(\tau_t^k, z^w, I), \phi \in [s, r, R])$



The architecture of SPLT Transformer for generating a reconstruction prediction

SPLT Transformer (III) – Planning

● Candidate trajectory generation

- The goal of this phase is to predict a possible continuation of that trajectory over the planning horizon h at current state s_t and stored history of the last k steps of the trajectory
 - $\hat{\tau}_{t-k}^{k+h} = \{s_{t-k}, a_{t-k}, \dots, s_t, \hat{a}_t, \hat{s}_{t+1}, \dots, \hat{s}_{t+h}, \hat{a}_{t+h}\}$
- The authors alternatively make autoregressive predictions from the policy and world models to predict these quantities
 - $\hat{a}_{t+i} = f_{\pi}(\{s_{t-k}, a_{t-k}, \dots, s_t, \hat{a}_t, \dots, \hat{s}_{t+i}\}, z^{\pi}) \rightarrow \hat{s}_{t+i+1}, \hat{r}_{t+i}, \hat{R}_{t+i+1} = f_w(\{s_{t-k}, a_{t-k}, \dots, s_t, \hat{a}_t, \dots, \hat{s}_{t+i}, \hat{a}_{t+i}\}, z_w)$
- They repeat this alternating procedure until reaching the horizon length h and compute $\hat{\tau}_t^h$ and its corresponding $\hat{R}(\hat{\tau}_t^h)$

● Action selection

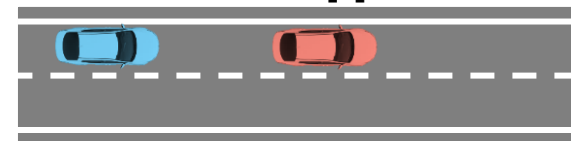
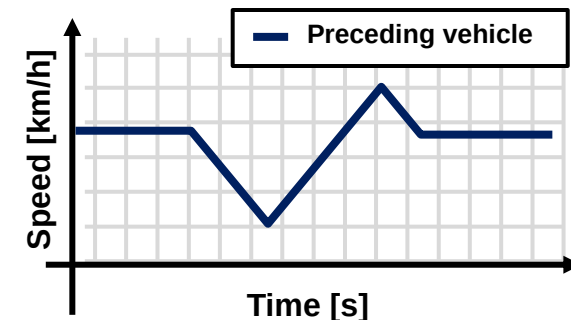
- Thanks to discrete latent variables, SPLT can enumerate all possible combinations of z^{π} and z^w
 - In the action selection phase, 256 different combinations of latent variables ($c = 2, n_w \leq 4$, and $n_{\pi} \leq 4$) only need to be considered
- In the trajectories, the authors selected the best trajectory that corresponds to
 - $\max_i \min_j \hat{R}_{ij}$, where $i \in [1, c^{n_{\pi}}]$, and $j \in [1, c^{n_w}]$
 - The intuition behind this procedure is that the SPLT is trying to pick policy to follow that will be robust to nay realistic possible future in the current environment
- They executed the first action of $\hat{\tau}_{i^*j^*}$ and repeat this procedure at every timestep

Experiments

Experiment environment

● Illustrative example: toy autonomous driving problem

- Vehicle control problem in car-following situation
 - Half of the time the leading vehicle will begin **hard-braking** at the last possible moment (about 70m)
 - The other half of the time the leading vehicle will **immediately speed up** to the maximum speed
 - Assumption that the perception and localization systems are well-built



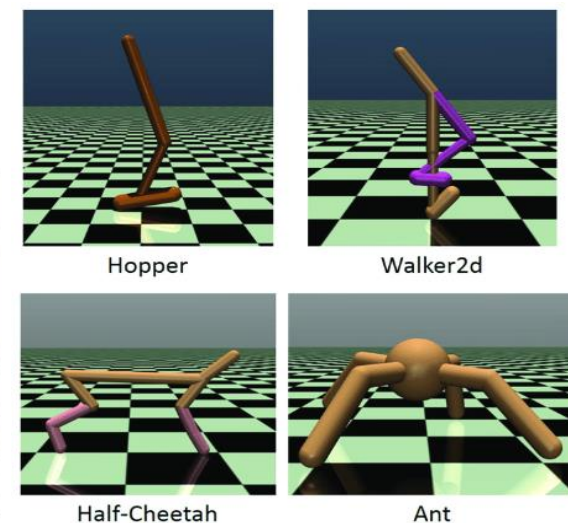
Simulation scenario

● Environment

- Collected dataset (~100000 steps) with a distribution of different IDM in simulation env
 - NoCrash env (based on Carla)
- Benchmark dataset
 - D4RL
 - HalfCheetah
 - Hopper
 - Walker2d



NoCrash RL env with Carla simulation environment



D4RL

Experiment results (I)

● Comparison with previous methods in Offline RL tasks

- Experiments showed comparable performance compared to previous SOTA methods in D4RL offline RL benchmark(HalfCheetah, Hopper, Walker2d)
 - **Imitation Learning**: Behavior Cloning(BC)
 - **Offline RL**: Model-Based Offline Planning(MBOP), Conservative Q-Learning(CQL), DT, TT
 - **Model-free RL**: Implicit Q-Learning(IQL)
- The authors describe that the reason for the low performance in the med-replay dataset setting was that the dataset contains a limited number of temporally consistent behaviors

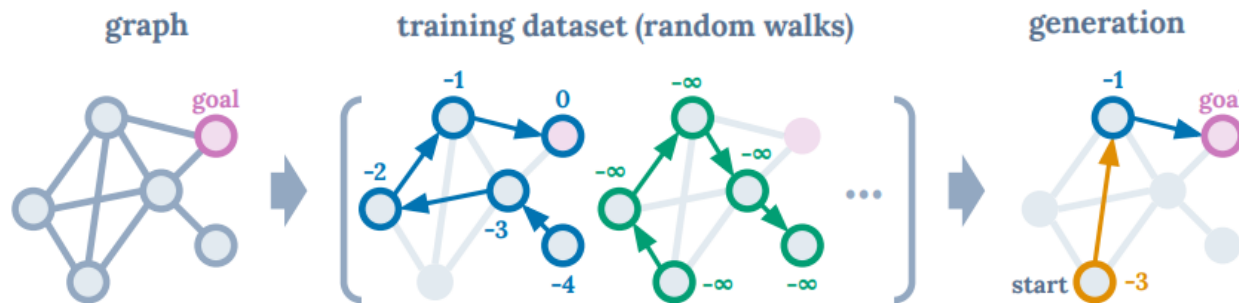
Dataset	Environment	BC	MBOP	CQL	DT	TT	IQL	SPLT(ours)
Med-Expert	HalfCheetah	59.9	105.9	91.6	86.8	95.0 $\pm$ 0.2	86.7	91.8 $\pm$ 0.5
Med-Expert	Hopper	79.6	55.1	105.4	107.6	110.0 $\pm$ 2.7	91.5	104.8 $\pm$ 2.6
Med-Expert	Walker2d	36.6	70.2	108.8	108.1	101.9 $\pm$ 6.8	109.6	108.6 $\pm$ 1.1
Medium	HalfCheetah	43.1	44.6	44.0	42.6	46.9 $\pm$ 0.4	47.4	44.3 $\pm$ 0.7
Medium	Hopper	63.9	48.8	58.5	67.6	61.1 $\pm$ 3.6	66.3	53.4 $\pm$ 6.5
Medium	Walker2d	77.3	41.0	72.5	74.0	79.0 $\pm$ 2.8	78.3	77.9 $\pm$ 0.3
Med-Replay	HalfCheetah	4.3	42.3	45.5	36.6	41.9 $\pm$ 2.5	44.2	42.7 $\pm$ 0.3
Med-Replay	Hopper	27.6	12.4	95.0	82.7	91.5 $\pm$ 3.6	94.7	75.0 $\pm$ 23.8
Med-Replay	Walker2d	36.9	9.7	77.2	66.6	82.6 $\pm$ 6.9	73.9	57.7 $\pm$ 4.7
	Average	47.7	47.8	77.6	74.7	78.9	76.9	72.9

Offline RL results in tabular form

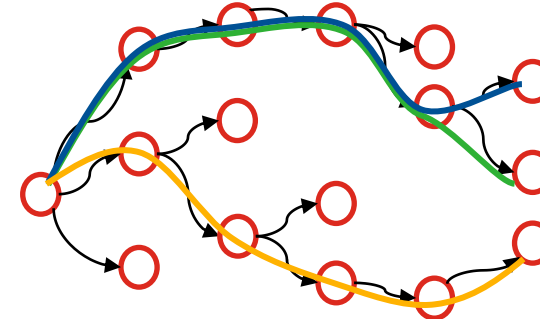
Experiment results (II) – Learning behavior for self-driving vehicle

● Qualitative analysis in complex stochastic task

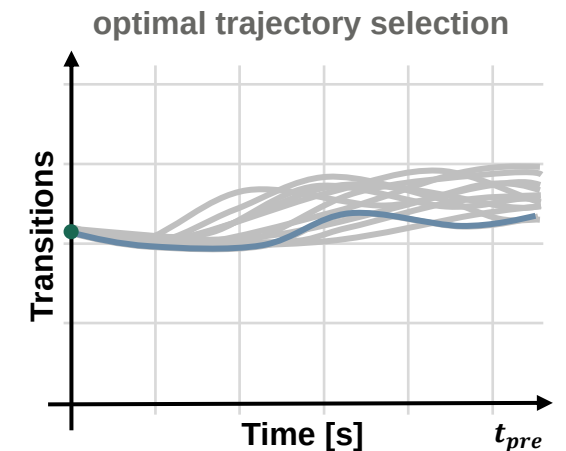
- Decision transformer and trajectory transformer are underperformed
 - For DT, the authors found that conditioning on the maximum return in the dataset leads to crashes every time the leading vehicle brakes
 - For TT, they found that the results depend heavily on the scope of the search used.
- SPLT Transformer achieved significant results
 - They insist that their world VAE was able to predict both possible modes for the leading vehicle's behavior and the policy VAE seems to be able to predict a range of different trailing behaviors



Return query in DT



Beam search in TT



Best trajectory selection in SPLT

Experiment results (III) – Learning behavior for self-driving vehicle

● Quantitative results

- Experiments showed comparable performance compared to previous SOTA methods
 - DT(m): is DT conditioned on the maximum return in the dataset
 - DT(e): is DT conditioned on the expected return of the best controller
 - DT(t): is DT with a hand-tuned conditional return
 - TT(a): is TT with more aggressive search parameters
 - IDM(t): is the best controller from the distribution used to collect the data
- They also evaluate the methods in the unseen dataset
 - SPLT outperformed the previous Offline RL methods in complex env
 - SPLT underperformed compared with IQN

Metric	SPLT (Ours)	BC	TT	DT(m)	DT(t)	IQL	Autopilot(t)
Success (%)	96.3 $\pm$ 2.1	95.3 $\pm$ 0.4	89.0 $\pm$ 2.8	92.0 $\pm$ 7.1	96.0 $\pm$ 3.3	99.0 $\pm$ 0.8	100.0
Speed (m/s)	2.71 $\pm$ 0.04	2.51 $\pm$ 0.0	2.67 $\pm$ 0.1	2.72 $\pm$ 0.03	2.72 $\pm$ 0.02	2.84 $\pm$ 0.09	2.75

Training results in tabular form

Metric	SPLT (Ours)	BC	DT(m)	DT(e)	DT(t)	TT	TT(a)	IDM(t)
Return	78.5 $\pm$ 0.2	58.0 $\pm$ 1.0	28.5 $\pm$ 0.4	68.9 $\pm$ 0.4	73.4 $\pm$ 0.8	58.8 $\pm$ 0.5	67.3 $\pm$ 5.6	78.6
Success (%)	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	50.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	100.0 $\pm$ 0.0	91.7 $\pm$ 5.8	100.0

Evaluation results in unseen routes

Thank you!

Q&A